

南開大學

信息隐藏技术课程实验报告

实验八：变换域隐藏法



学 院 密码与网络空间安全学院
专 业 信息安全、法学双学位班
学 号 2313815
姓 名 段俊宇
班 级 信息安全、法学双学位班

一、实验要求

1、 DCT 或 DWT 域的数字水印包括：修改系数方法和系数比较方法。以上两种变换域或方法任选一种, 实现变换域中的信息隐藏与提取。

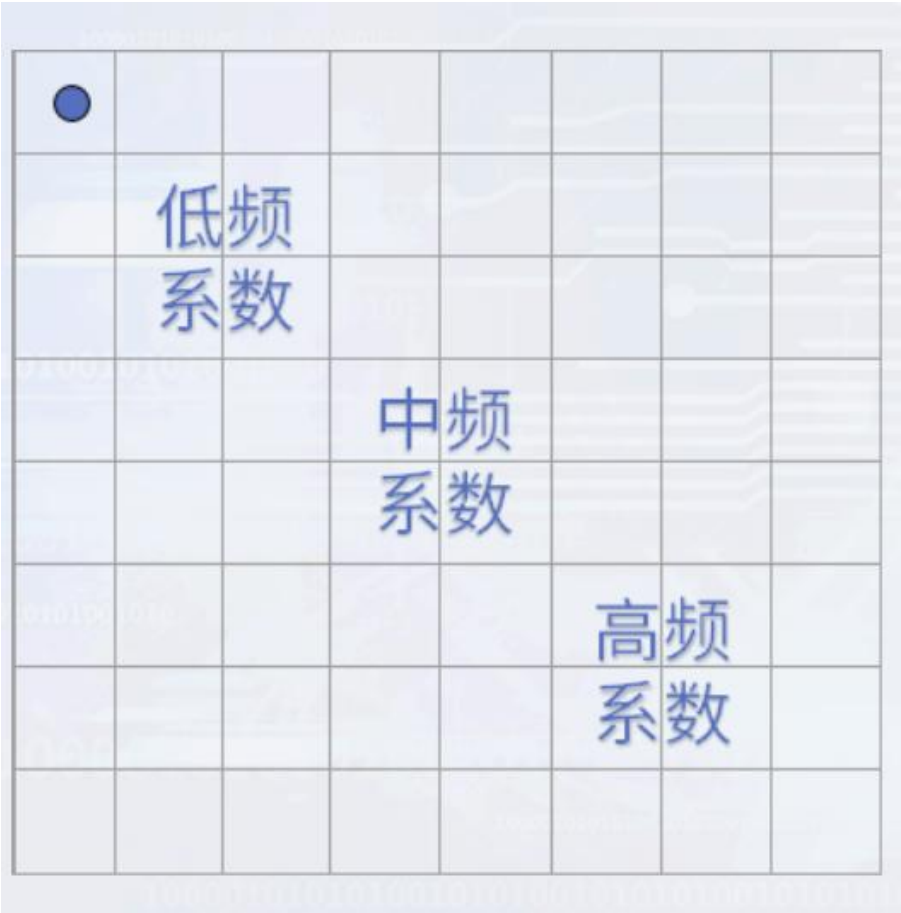
二、实验原理

变换域技术:在载体的显著区域隐藏信息,比 LSB 方法能够更好地抵抗攻击,而且还保持了对人类感观的不可察觉性。常用的变换域方法有离散余弦变换 (DCT)、离散小波变换 (DWT)、离散傅立叶变换 (DFT)。

1. DCT 域的隐写

图像压缩标准 (JPEG) 的核心是二维 DCT 变换, 在 DCT 域中的隐写, 可以有效地抵抗 JPEG 有损压缩。

图像分为 8×8 的像素块, 进行二维 DCT 变换, 得到 8×8 的 DCT 系数。最左上角的那个系数为直流系数, 其余为交流系数。左上角的部分为直流和低频, 右下角的部分为高频, 中间区域为中频。中低频系数包含了图像的大部分能量, 是对人的视觉最重要的部分。



算法三：不需要原始载体的隐写方法，利用载体中两个特定数的相对大小来代表隐藏的信息。

嵌入：载体图像分为 8×8 的块，做二维 DCT 变换，伪随机地选择一个图像块 B1，分别选择其中的两个位置，比如用 (u_1, v_1) 和 (u_2, v_2) 代表所选定的两个系数的坐标。如果 $B_1(u_1, v_1) > B_1(u_2, v_2)$ ，代表隐藏 1；如果相反，则交换两系数。如果 $B_1(u_1, v_1) < B_1(u_2, v_2)$ ，代表隐藏 0；如果相反，则交换两系数。最后做二维 DCT 逆变换。

提取：接收者进行二维 DCT 变换，比较每一块中约定位置的 DCT 系数值，根据其相对大小，得到隐藏信息的比特串，从而恢复出秘密信息。

特点：不需原始图像。

注意：如果选定位置的两个系数相差太大，则对图像影响较大。应选择相近的值(如中频系数)。

算法四：算法三的扩展，利用 DCT 中频系数中的三个系数之间的相对关系来对秘密信息进行编码。

嵌入：选择三个位置 (u_1, v_1) 、 (u_2, v_2) 、 (u_3, v_3) 。

- 嵌入 1：令 $B_i(u_1, v_1) > B_i(u_3, v_3) + D$ ， $B_i(u_2, v_2) > B_i(u_3, v_3) + D$ 。
- 嵌入 0：令 $B_i(u_1, v_1) < B_i(u_3, v_3) - D$ ， $B_i(u_2, v_2) < B_i(u_3, v_3) - D$ 。

如果数据不符，则修改这三个系数值，使得它们满足上述关系。其中参数 D 的选择要考虑隐藏的健壮性和不可察觉性之间的平衡，D 越大，隐藏算法对于图像处理就越健壮，但是对图像的改动就越大，越容易引起察觉。

如果需要做的修改太大,则放弃该块,将其标识为“无效”。对这三个系数做小量的修改使得它们满足下面条件之一：

- $B_i(u_1, v_1) \leq B_i(u_3, v_3) \leq B_i(u_2, v_2)$
- $B_i(u_2, v_2) \leq B_i(u_3, v_3) \leq B_i(u_1, v_1)$

提取：对图像进行 DCT 变换，比较每一块相应三个位置的系数，从它们之间的关系，可以判断隐藏的是信息“1”、“0”还是“无效”块，这样就可以恢复秘密信息。

2. DWT 域的隐写

二维小波分解：一级小波分解后得到如下的四个部分，图像的主要能量

集中在低频近似部分。

- 左上：低频近似部分。
- 左下：垂直方向细节部分。
- 右上：水平方向细节部分。
- 右下：对角线方向细节部分。

DCT 域和 DWT 域的隐写方法有修改系数方法和系数比较方法。

三、实验环境

Matlab_R2025b, Image Processing Toolbox

四、实验过程

1. 参数设置和图像准备

如下所示，我将使用注释的方式对代码进行解释。

```
1. % 参数配置
2. BLOCK_SIZE = 8;
3.
4. % 按 Zigzag 顺序取前 28 个中低频系数
5. COEF_POS = [
6.     1,2;  2,1;
7.     1,3;  2,2;  3,1;
8.     1,4;  2,3;  3,2;  4,1;
9.     1,5;  2,4;  3,3;  4,2;  5,1;
10.    1,6;  2,5;  3,4;  4,3;  5,2;  6,1;
11.    1,7;  2,6;  3,5;  4,4;  5,3;  6,2;  7,1;
12.    1,8;
13. ];
14.
15. n_coef_total = size(COEF_POS, 1);
16. fprintf('使用 %d 个系数位置 (Zigzag 中低频) \n', n_coef_total);
17. Q = 8;      % QIM 量化步长
18.
19. % 载入载体图像
20. cover_rgb = imread('Picture\1.bmp');
21. if size(cover_rgb, 3) == 3
22.     cover_gray = rgb2gray(cover_rgb);
23. else
24.     cover_gray = cover_rgb;
25. end
26.
```

```

27. % 裁剪为 BLOCK_SIZE 的整数倍
28. [H, W] = size(cover_gray);
29. H = floor(H / BLOCK_SIZE) * BLOCK_SIZE;
30. W = floor(W / BLOCK_SIZE) * BLOCK_SIZE;
31. cover_gray = cover_gray(1:H, 1:W);
32.
33. fprintf('===== DCT-QIM 图像隐写 =====\n');
34. fprintf('载体图像尺寸: %d x %d\n', H, W);
35.
36. % 准备秘密图像
37. secret_img = imread('Picture\2.bmp');
38. if size(secret_img, 3) == 3
39.     secret_gray = rgb2gray(secret_img);
40. else
41.     secret_gray = secret_img;
42. end
43.
44. [secret_h, secret_w] = size(secret_gray);
45.
46. fprintf('秘密图像尺寸: %d x %d\n', secret_h, secret_w);
47.
48. % 将秘密图像转换为比特流
49. secret_bits = img2bits(secret_gray);
50. msg_len = length(secret_bits);
51.
52. % 构造 32 位高度头 + 32 位宽度头 + 秘密比特
53. height_bits = bitget(uint32(secret_h), 32:-1:1);
54. width_bits = bitget(uint32(secret_w), 32:-1:1);
55. payload = [height_bits, width_bits, secret_bits];
56.
57. fprintf('秘密图像比特数: %d bits\n', msg_len);
58. fprintf('含头的总载荷: %d bits\n\n', length(payload));

```

这部分代码是参数设置和图像准备。参数中包括 DCT 分块大小、按 Zigzag 顺序选取的中低频系数位置以及 QIM 量化步长，之后将载体图片裁剪为 DCT 分块的整数倍方便嵌入，将秘密图像转换为比特流并构造载荷。由于 DCT 和 Zigzag 是结合使用的，因此按视觉重要性排序 DCT 系数，选取前 28 个中低频系数嵌入，这样可以保证图像的完整性。

将图像转换为比特流的函数如下所示：

```

1. % 将图像转换为二进制序列
2. function bits = img2bits(img)
3.     img_uint8 = uint8(img);

```

```

4.     [h, w] = size(img_uint8);
5.     bits = zeros(1, h * w * 8, 'uint8');
6.     bit_idx = 1;
7.
8.     for i = 1:h
9.         for j = 1:w
10.            pixel_bits = bitget(img_uint8(i, j), 8:-1:1);
11.            bits(bit_idx:bit_idx+7) = pixel_bits;
12.            bit_idx = bit_idx + 8;
13.        end
14.    end
15. end

```

2. 图像的嵌入

如下所示，我将使用注释的方式对代码进行解释。

```

1. function stego_img = dct_qim_embed(cover_img, secret_bits, block_size, coef_pos, Q)
2. cover_img = double(cover_img);
3. [H, W] = size(cover_img);
4. nbh = floor(H / block_size);
5. nbw = floor(W / block_size);
6. n_coef = size(coef_pos, 1); % 系数位置数量
7. max_bits = nbh * nbw * n_coef; % 总容量 = 块数 × 每块系数数
8.
9. fprintf('图像尺寸: %d x %d | 分块: %dx%d = %d 块\n', H, W, nbh, nbw, nbh*nbw);
10. fprintf('每块嵌入: %d bits | 总容量: %d bits | 待嵌入: %d bits\n', n_coef, max_bits,
length(secret_bits));
11. if length(secret_bits) > max_bits
12.     error('秘密信息过长! 最大 %d bits', max_bits);
13. end
14.
15. stego = cover_img;
16. bit_idx = 1;
17. n_bits = length(secret_bits);
18.
19. for bh = 1:nbh
20.     for bw = 1:nbw
21.         rs = (bh-1)*block_size+1; cs = (bw-1)*block_size+1;
22.         block = stego(rs:rs+block_size-1, cs:cs+block_size-1);
23.         D = dct2(block);
24.
25.         % 在当前块中嵌入多个比特（使用多个系数位置）
26.         for coef_idx = 1:n_coef
27.             if bit_idx > n_bits, break; end
28.

```

```

29.         r = coef_pos(coef_idx, 1);
30.         c = coef_pos(coef_idx, 2);
31.
32.         idx = round(D(r,c) / Q);    % 量化索引
33.         bit = double(secret_bits(bit_idx));
34.
35.         if bit == 0
36.             if mod(idx,2) ~= 0, idx = idx + 1; end    % 偶数索引 → bit=0
37.         else
38.             if mod(idx,2) == 0, idx = idx + 1; end    % 奇数索引 → bit=1
39.         end
40.         D(r,c) = idx * Q;            % 反量化写回
41.         bit_idx = bit_idx + 1;
42.     end
43.
44.     stego(rs:rs+block_size-1, cs:cs+block_size-1) = idct2(D);
45.     if bit_idx > n_bits, break; end
46. end
47. if bit_idx > n_bits, break; end
48. end
49.
50. stego_img = uint8(max(0, min(255, round(stego))));
51. fprintf('嵌入完成! 共嵌入 %d bits\n', n_bits);
52. end

```

这部分代码实现了对图像的 QIM 嵌入。该嵌入方法使用量化索引调制，根据已经分好的块，对每个块进行 DCT 变换，然后修改 DCT 系数。对于比特 0，设置为偶数量化索引；对于比特 1，设置为奇数量化索引。最后逆 DCT 变换重建含密图像。

3. 图像的提取

如下所示，我将使用注释的方式对代码进行解释。

```

1. % 提取函数: QIM 解调
2. function secret_bits = dct_qim_extract(stego_img, num_bits, block_size, coef_pos, Q)
3. stego_img = double(stego_img);
4. [H, W]    = size(stego_img);
5. nbh = floor(H / block_size);
6. nbw = floor(W / block_size);
7. n_coef = size(coef_pos, 1); % 系数位置数量
8.
9. secret_bits = zeros(1, num_bits, 'uint8');
10. bit_idx = 1;
11.

```



```

12. for bh = 1:nbh
13.     for bw = 1:nbw
14.         rs = (bh-1)*block_size+1; cs = (bw-1)*block_size+1;
15.         block = stego_img(rs:rs+block_size-1, cs:cs+block_size-1);
16.         D = dct2(block);
17.
18.         % 从当前块中提取多个比特
19.         for coef_idx = 1:n_coef
20.             if bit_idx > num_bits, break; end
21.
22.             r = coef_pos(coef_idx, 1);
23.             c = coef_pos(coef_idx, 2);
24.
25.             idx = round(D(r,c) / Q);          % 量化索引
26.             secret_bits(bit_idx) = uint8(mod(abs(idx), 2)); % 奇偶 → bit
27.             bit_idx = bit_idx + 1;
28.         end
29.         if bit_idx > num_bits, break; end
30.     end
31.     if bit_idx > num_bits, break; end
32. end
33.
34. fprintf('提取完成! 共提取 %d bits\n', num_bits);
35. end
36.
37. % 提取
38. fprintf('\n--- 开始提取 ---\n');
39. extracted_all = dct_qim_extract(stego_img, length(payload), BLOCK_SIZE, COEF_POS,
Q);
40.
41. % 解析 32 位高度头
42. height_u32 = uint32(0);
43. for i = 1:32
44.     height_u32 = bitor(bitshift(height_u32, 1), uint32(extracted_all(i)));
45. end
46. extracted_h = double(height_u32);
47.
48. % 解析 32 位宽度头
49. width_u32 = uint32(0);
50. for i = 33:64
51.     width_u32 = bitor(bitshift(width_u32, 1), uint32(extracted_all(i)));
52. end
53. extracted_w = double(width_u32);
54.

```

```

55. fprintf('解析到的秘密图像尺寸: %d x %d\n', extracted_h, extracted_w);
56.
57. % 安全检查
58. max_extractable = length(extracted_all) - 64;
59. expected_bits = extracted_h * extracted_w * 8;
60. if expected_bits < 1 || expected_bits > max_extractable
61.     error(['尺寸头解析异常! 预期比特数=%d, 最大可提取=%d bits'], expected_bits,
max_extractable);
62. end
63.
64. % 提取秘密比特
65. msg_bits_ex = extracted_all(65 : 64 + expected_bits);
66.
67. % 将比特流还原为图像
68. extracted_img = bits2img(msg_bits_ex, extracted_h, extracted_w);
69.

```

这部分代码实现了对图像的提取。在提取时先对含密图像块进行 DCT 变换，量化系数得到索引并根据索引的奇偶性判断秘密比特。这样处理之后对高度头和宽度头进行解析，提取秘密比特并将其还原为秘密图像。

比特流还原为图像的函数如下所示：

```

1. % 将二进制序列还原为图像
2. function img = bits2img(bits, h, w)
3.     img = zeros(h, w, 'uint8');
4.     bit_idx = 1;
5.
6.     for i = 1:h
7.         for j = 1:w
8.             byte_val = uint8(0);
9.             for b = 1:8
10.                 byte_val = bitor(bitshift(byte_val, 1), uint8(bits(bit_idx)));
11.                 bit_idx = bit_idx + 1;
12.             end
13.             img(i, j) = byte_val;
14.         end
15.     end
16. end

```

4. 图像质量评估

如下所示，我将使用注释的方式对代码进行解释。

```

1. % 图像质量评估
2. cover_d = double(cover_gray);

```

```

3. stego_d = double(stego_img);
4. mse = mean((cover_d(:) - stego_d(:)).^2);
5. if mse < 1e-10
6.     psnr_val = Inf;
7.     fprintf('\n 警告: PSNR=Inf, 系数未被修改! 请增大 Q 值或换用其他系数位置.\n');
8. else
9.     psnr_val = 10 * log10(255^2 / mse);
10. end
11. try
12.     ssim_val = ssim(stego_img, cover_gray);
13. catch
14.     ssim_val = NaN;
15. end
16.
17. fprintf('\n===== 图像质量评估 =====\n');
18. fprintf('MSE : %.4f\n', mse);
19. if isinf(psnr_val)
20.     fprintf('PSNR: Inf dB\n');
21. else
22.     fprintf('PSNR: %.2f dB\n', psnr_val);
23. end
24. if ~isnan(ssim_val)
25.     fprintf('SSIM: %.6f\n', ssim_val);
26. end

```

这部分代码实现了对含密图像的质量评估，通过计算 PSNR 和 SSIM 展示图像质量。

5. 可视化展示结果

```

1. % 可视化
2. figure('Name','DCT-QIM 图像隐写','NumberTitle','off','Position',[80,80,1200,700]);
3.
4. subplot(2,3,1); imshow(cover_gray);
5. title('载体图像 Cover','FontSize',12);
6. xlabel(sprintf('%dx%d',W,H));
7.
8. subplot(2,3,2); imshow(stego_img);
9. if isinf(psnr_val)
10.     title('含密图像 Stego PSNR=Inf','FontSize',12);
11. else
12.     title(sprintf('含密图像 Stego\nPSNR=%.2f dB',psnr_val),'FontSize',12);
13. end
14.
15. diff_img = abs(cover_d - stego_d);

```

```

16. subplot(2,3,3); imshow(diff_img*10,[]); colormap(gca,hot);
17. title(sprintf('差值×10 (max diff=%d)',round(max(diff_img(:)))),'FontSize',12);
18.
19. subplot(2,3,4); imshow(secret_gray);
20. title('原始秘密图像','FontSize',12);
21. xlabel(sprintf('%dx%d',secret_w,secret_h));
22.
23. subplot(2,3,5); imshow(extracted_img);
24. title(sprintf('提取的秘密图像\nPSNR=%.2f dB',psnr_secret),'FontSize',12);
25.
26. % 保存
27. imwrite(stego_img, 'Picture/stego_output.bmp');
28. imwrite(extracted_img, 'Picture/extracted_secret.bmp');
29. fprintf('\n 含密图像已保存: Picture/stego_output.bmp\n');
30. fprintf('提取的秘密图像已保存: Picture/extracted_secret.bmp\n');

```

这部分代码是对上述功能结果的展示，并将嵌入和提取的图片进行保存。

接下来运行代码，结果如下所示：



```
使用 28 个系数位置 (Zigzag中低频)
===== DCT-QIM 图像隐写 =====
载体图像尺寸: 1152 x 2544
秘密图像尺寸: 396 x 400
秘密图像比特数: 1267200 bits
含头的总载荷: 1267264 bits

--- 开始嵌入 (Q=8) ---
图像尺寸: 1152 x 2544 | 分块: 144x318 = 45792块
每块嵌入: 28 bits | 总容量: 1282176 bits | 待嵌入: 1267264 bits
嵌入完成! 共嵌入 1267264 bits

===== 图像质量评估 =====
MSE : 18.1692
PSNR: 35.54 dB
SSIM: 0.882680

--- 开始提取 ---
提取完成! 共提取 1267264 bits
解析到的秘密图像尺寸: 396 x 400

===== 提取结果 =====
提取图像与原始秘密图像PSNR: 49.98 dB
✓ 图像提取成功!

含密图像已保存: Picture/stego_output.bmp
提取的秘密图像已保存: Picture/extracted_secret.bmp
```

从上面的结果可以看出，成功完成了嵌入和提取，并且还原了原始的南开校徽二值图。在可视化结果中还展示了差值图像，也就是载体图像与含密图像对应像素值的绝对差，亮斑显示秘密信息嵌入的具体位置，差值越小不可感知性越好。最后的含密图像用肉眼看不出区别和亮斑，说明嵌入没有问题，这样就成功完成了本次实验！

五、实验结论及心得体会

本次实验中我使用 DCT 的修改系数方法实现了对图像的嵌入和提取。在实验过程中我理解了 DCT 变换域隐藏的流程，巩固了课上的知识。此外，我还了解到 QIM 方法相对传统的修改系数法更加鲁棒，因为它利用量化的离散特性，对噪声和压缩有更好的抵抗能力。